

T.C.
YILDIZ TEKNİK ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ



WAVELET TABANLI GÖRÜNTÜ SIKIŞTIRMA

Muhtalip DEDE
24526003

FİNAL PROJESİ
Matematik Mühendisliği Anabilim Dalı
Matematik Mühendisliği Programı

Dersi Veren
Prof. Dr. İbrahim EMİROĞLU

Haziran, 2026

İÇİNDEKİLER

ŞEKİL LİSTESİ	iii
TABLO LİSTESİ	iv
ÖZET	v
ABSTRACT	vi
1 GİRİŞ	1
2 DWT, Huffman Kodlama ve Algoritmik İyileştirmeler	3
2.1 Ayrık Wavelet Dönüşümü (DWT)	3
2.1.1 1B DWT	3
2.1.2 2B DWT	4
2.1.3 Wavelet Türleri	5
2.2 Huffman Kodlama	5
2.3 Katsayı Kuantizasyonu	6
2.4 Thresholding (Eşikleme)	7
2.4.1 Hard Thresholding	7
2.4.2 Soft Thresholding	7
2.5 DWT ve Huffman Birlikte Kullanımı	7
2.6 Değerlendirme Metrikleri	7
2.6.1 PSNR	7
2.6.2 Sıkıştırma Oranı	8
3 Renkli Görüntü Temsili, YCbCr ve OpenCV Kullanımı	9
3.1 Dijital Görüntü Temsili	9
3.2 RGB Kanal Bazlı Sıkıştırma	9
3.3 YCbCr Renk Uzayı	10
3.4 OpenCV Kullanımı	11
3.4.1 Görüntü Okuma	11
3.4.2 Kanal Ayırıştırma	11
3.4.3 Görselleştirme	11

3.5	OpenCV'nin Projeye Katkısı	11
4	Sistem Tasarımı	13
4.1	Genel Sistem Mimarisi	13
4.2	Sıkıştırma Süreci	13
4.3	Veri Akış Diyagramı	14
4.4	Parametre Yönetimi	16
4.5	Modüler Tasarımın Avantajları	17
4.6	Performans Kriterleri	17
4.7	Kaynak Kod Deposu	17
5	Tartışma	18
5.1	Wavelet Seçimi	18
5.2	Decomposition Seviyesi	18
5.3	Renk Uzayı Karşılaştırması	19
5.4	Algoritmik İyileştirmelerin Birlikte Etkisi	19
5.5	JPEG ile Karşılaştırma	19
5.6	Sistem Sınırlamaları	20
5.7	Kalite–Sıkıştırma Dengesi	20
6	Test Sonuçları ve Performans Analizi	21
6.1	Test Ortamı ve Deney Protokolü	21
6.2	Sabit Deney Parametreleri	21
6.3	D1: Wavelet Türü Karşılaştırması	22
6.3.1	D1 Sonuçlarının Yorumu	22
6.4	D2: Decomposition Seviyesi Karşılaştırması	23
6.4.1	D2 Sonuçlarının Yorumu	23
6.5	D3: Gri ve Renkli Görüntü Karşılaştırması	24
6.5.1	D3 Sonuçlarının Yorumu	24
6.6	D4: RGB ve YCbCr Karşılaştırması	25
6.6.1	D4 Sonuçlarının Yorumu	25
6.7	Tüm Deneylerin Özet Tablosu	26
6.8	Deneylerin Tekrarlanması	26
7	Sonuç	28
	KAYNAKÇA	29

ŞEKİL LİSTESİ

Şekil 6.1	Wavelet karşılaştırması: HAAR (sol), SYM4 (orta), DB2 (sağ) . .	23
Şekil 6.2	Seviye karşılaştırması: Level 1 (sol), Level 2 (orta), Level 3 (sağ)	24
Şekil 6.3	Gri (sol), RGB (orta) ve YCbCr (sağ) rekonstrüksiyon karşılaştırması	25
Şekil 6.4	RGB (sol) ve YCbCr (sağ) rekonstrüksiyon karşılaştırması	26

TABLO LİSTESİ

Tablo 2.1	Projede Kullanılan Wavelet Türleri	5
Tablo 4.1	Sistem Modülleri	13
Tablo 6.1	Wavelet Türü Karşılaştırması (lena.bmp, Level=2, Q=10, T=5)	22
Tablo 6.2	Decomposition Seviyesi Karşılaştırması (lena.bmp, SYM4, Q=10, T=5)	23
Tablo 6.3	Gri ve Renkli Görüntü Karşılaştırması (SYM4, Level=2, Q=10, T=5)	24
Tablo 6.4	RGB ve YCbCr Karşılaştırması (lena.png, SYM4, Level=2) .	25
Tablo 6.5	Tüm DeneySEL Sonuçların Özeti (wavelet_experiments) .	26

WAVELET TABANLI GÖRÜNTÜ SIKIŞTIRMA

Muhtalip DEDE

24526003

Matematik Mühendisliği Anabilim Dalı

Danışman: Prof. Dr. İbrahim EMİROĞLU

Bu Final Projesi kapsamında, Huffman kodlama ile birlikte çalışan wavelet tabanlı bir görüntü sıkıştırma sistemi C++ ve OpenCV kullanılarak geliştirilmiştir. Sistem; çok seviyeli 2B DWT, kuantizasyon, eşikleme ve entropi kodlama adımlarını gri ve renkli görüntülere uygular. Renkli işleme için RGB kanal bazlı ve YCbCr tabanlı (algısal profil) yaklaşımlar desteklenmektedir.

lena.bmp ve lena.png üzerinde PSNR ve sıkıştırma oranı ile yapılan değerlendirmede HAAR en yüksek PSNR'ı (39.80 dB), SYM4 + level 2 ise en iyi dengeyi (19.37 dB, 3.96x) sağlamıştır. Renkli görüntüde RGB modu 3.75x, YCbCr modu ise parlaklık kanalını koruyarak 19.68 dB PSNR elde etmiştir. Sistem; renkli görüntü desteği, algoritmik iyileştirmeler ve deneysel analiz gereksinimlerini karşılamaktadır.

Anahtar Kelimeler: Wavelet dönüşümü, DWT, Huffman kodlama, görüntü sıkıştırma, YCbCr, PSNR, OpenCV

ABSTRACT

WAVELET BASED IMAGE COMPRESSION

Muhtalip DEDE
24526003

Department of Mathematical Engineering
Final Project

This Final Project presents a wavelet-based image compression system with Huffman coding, implemented in C++ using OpenCV. The system applies multi-level 2D DWT, quantization, thresholding, and entropy coding to grayscale and color images. Color support includes RGB channel-wise and YCbCr-based compression with perceptually weighted channel profiles.

Performance was evaluated on `lena.bmp` and `lena.png` using PSNR and compression ratio across four wavelet types, decomposition levels, and color modes. HAAR achieved the highest PSNR (39.80 dB); SYM4 with level 2 offered the best balance (19.37 dB, 3.96x). RGB mode achieved 3.75x compression on color images; YCbCr preserved the luminance channel with similar PSNR (19.68 dB). The system meets the project requirements for color processing, algorithmic improvements, and experimental analysis.

Keywords: Wavelet transform, DWT, Huffman coding, image compression, YCbCr, PSNR, OpenCV

YILDIZ TECHNICAL UNIVERSITY
GRADUATE SCHOOL OF SCIENCE AND ENGINEERING

1 GİRİŞ

Dijital görüntülerin depolanması, iletilmesi ve işlenmesi; yüksek çözünürlük, artan veri hacmi ve sınırlı bant genişliği nedeniyle giderek daha kritik bir mühendislik problemi haline gelmiştir. Özellikle mobil cihazlar, tıbbi görüntüleme sistemleri, uzaktan algılama ve multimedya uygulamalarında veri boyutunun azaltılması hem maliyet hem de performans açısından büyük önem taşımaktadır.

Görüntü sıkıştırma teknikleri genel olarak kayıplı (lossy) ve kayıpsız (lossless) olmak üzere iki ana grupta incelenir. Kayıpsız yöntemler orijinal veriyi tamamen geri oluşturabilirken, kayıplı yöntemler belirli bir kalite kaybını kabul ederek çok daha yüksek sıkıştırma oranları elde edebilir. JPEG standardı gibi yaygın yöntemler DCT tabanlı kayıplı sıkıştırmaya dayanırken, JPEG 2000 standardı wavelet tabanlı dönüşüm yaklaşımını temel alır [1].

Bu Final Projesi kapsamında, verilen temel C++ uygulaması analiz edilmiş ve wavelet tabanlı bir görüntü sıkıştırma sistemi geliştirilmiştir. Başlangıç implementasyonu yalnızca gri seviye görüntüler için 2B DWT ve Huffman kodlama içermektedir. Geliştirilen sistem ise aşağıdaki genişletmeleri sağlamaktadır:

- Renkli görüntü desteği (RGB ve YCbCr)
- Katsayı kuantizasyonu
- Hard/soft thresholding
- Çok seviyeli DWT
- PSNR ve sıkıştırma oranı metrikleri ile deneysel analiz

Wavelet dönüşümü, görüntüyü farklı frekans bantlarına ayırarak enerjinin az sayıda katsayıda toplanmasını sağlar [2]. Bu özellik, Huffman gibi entropi tabanlı kodlayıcıların daha verimli çalışmasına olanak tanır. Özellikle sıfıra yakın

katsayıların çokluğu, deęişken uzunluklu kodlama ile ciddi boyut kazancı sağlar [3].

Bu raporda sırasıyla kullanılan yöntemlerin teorik temelleri, sistem tasarımı, deneysel sonuçlar ve elde edilen bulgular sunulmaktadır. Amaç; hem proje gereksinimlerini karşılayan çalışır bir sistem geliştirmek hem de farklı wavelet türleri, ayrıştırma seviyeleri ve renk uzaylarının sıkıştırma performansına etkisini analiz etmektir.

DWT, Huffman Kodlama ve Algoritmik İyileştirmeler

2.1 Ayırık Wavelet Dönüşümü (DWT)

Ayrık Wavelet Dönüşümü (Discrete Wavelet Transform, DWT), bir sinyali veya görüntüyü farklı ölçeklerdeki frekans bileşenlerine ayıran çok çözünürlüklü bir dönüşümdür [4]. Fourier dönüşümünün aksine DWT, frekans bilgisinin yanı sıra uzamsal konum bilgisini de korur. Bu nedenle görüntü işleme uygulamalarında kenar, doku ve detay bilgilerinin ayrıştırılmasında etkilidir.

2.1.1 1B DWT

Bir boyutlu DWT işleminde giriş vektörü alçak geçiren (L_o) ve yüksek geçiren (H_i) filtreler ile konvolüsyona tabi tutulur. Elde edilen çıktının ilk yarısı yaklaşık (approximation) katsayıları, ikinci yarısı ise detay (detail) katsayılarını temsil eder.

Projede kullanılan `dwt1D` fonksiyonu, verilen filtre katsayıları ile periodik sınır koşulları altında 1B ayrıştırma gerçekleştirir. Ters dönüşüm için `idwt1D` fonksiyonu, rekonstrüksiyon filtreleri (L_o_R , H_i_R) kullanılarak uygulanır.

Listing 2.1’de `dwt1D` fonksiyonunun çekirdek döngüsü verilmiştir. Her i indeksi için alçak geçiren filtre (L_o) ile konvolüsyon sonucu yaklaşık katsayılar (`temp[i]`), yüksek geçiren filtre (H_i) ile konvolüsyon sonucu ise detay katsayıları (`temp[n/2 + i]`) elde edilir. $(2*i + k) \% n$ ifadesi periodik sınır koşulunu sağlar.

```

1 void dwt1D(vector<double>& data,
2           const vector<double>& Lo,
3           const vector<double>& Hi)
4 {
5     int n = data.size();
6     int f = Lo.size();
7     vector<double> temp(n, 0.0);
8
9     for (int i = 0; i < n / 2; i++) {
```

```

10     for (int k = 0; k < f; k++) {
11         int idx = (2 * i + k) % n;
12         temp[i] += data[idx] * Lo[k];
13         temp[n / 2 + i] += data[idx] * Hi[k];
14     }
15 }
16 data = temp;
17 }

```

Listing 2.1 1B DWT implementasyonu (wavelet.cpp)

2.1.2 2B DWT

İki boyutlu DWT, ayrılabilir (separable) yaklaşım ile uygulanır:

1. Her satır için 1B DWT
2. Her sütun için 1B DWT

Tek seviye 2B DWT sonucunda görüntü dört alt banda ayrılır:

- **LL:** Düşük frekans yaklaşım bileşeni
- **LH:** Yatay detay bileşeni
- **HL:** Dikey detay bileşeni
- **HH:** Çapraz detay bileşeni

Çok seviyeli DWT uygulandığında, bir sonraki seviyede yalnızca LL alt bandına tekrar dönüşüm uygulanır. Bu sayede daha yüksek frekanslı detaylar farklı ölçeklerde temsil edilir.

Listing 2.2’de dwt2D fonksiyonunun yapısı gösterilmektedir. Fonksiyon önce her satır için, ardından her sütun için dwt1D çağırır. `img.rows >> lev` ifadesi, her seviyede işlenen alt bölgenin boyutunu belirler (bit kaydırma ile yarıya indirme).

```

1 void dwt2D(Mat& img, const WaveletFilter& wf, int levels)
2 {
3     for (int lev = 0; lev < levels; lev++) {
4         int r = img.rows >> lev;
5         int c = img.cols >> lev;
6         for (int i = 0; i < r; i++) {

```

```

7         vector<double> row(c);
8         for (int j = 0; j < c; j++)
9             row[j] = img.at<double>(i, j);
10        dwtd1D(row, wf.Lo_D, wf.Hi_D);
11        for (int j = 0; j < c; j++)
12            img.at<double>(i, j) = row[j];
13    }
14    for (int j = 0; j < c; j++) { /* sutun bazli DWT */ }
15 }
16 }

```

Listing 2.2 2B DWT implementasyonu (wavelet.cpp)

2.1.3 Wavelet Türleri

Projede dört farklı wavelet ailesi desteklenmektedir:

Tablo 2.1 Projede Kullanılan Wavelet Türleri

Wavelet	Filtre Uzunluğu	Özellik
Haar	2	Basit, hızlı, blok artefakt eğilimi
DB2	4	Daubechies, daha yumuşak geçiş
DB4	8	Daha iyi frekans ayrımı
SYM4	8	Simetrik, görüntü işleme için uygun

2.2 Huffman Kodlama

Huffman kodlama, sembol frekanslarına göre değişken uzunluklu ikili kodlar üreten kayıpsız bir entropi kodlama yöntemidir [3]. Algoritma şu adımlarla çalışır:

1. Sembol frekansları hesaplanır.
2. Frekanslara göre min-heap (öncelik kuyruğu) oluşturulur.
3. En düşük frekanslı iki düğüm birleştirilerek Huffman ağacı kurulur.
4. Ağaç üzerinden her sembol için bit kodu türetilir.
5. Kodlar bit dizisine dönüştürülür ve 8-bit byte bloklarına paketlenir.

DWT sonrasında çoğu katsayı sıfıra yakın değerler aldığından, Huffman kodlayıcı bu sembollere kısa kodlar atayarak yüksek sıkıştırma oranı elde eder.

Listing 2.3’de Huffman agacinin olusturulmasi ve bit paketleme adimlari gosterilmektedir.

```

1 vector<uint8_t> huffmanEncode(const vector<int>& data, ...)
2 {
3     map<int, int> freq;
4     for (int v : data) freq[v]++;
5     priority_queue<HuffmanNode*, ..., NodeComparator> pq;
6     while (pq.size() > 1) {
7         HuffmanNode* a = pq.top(); pq.pop();
8         HuffmanNode* b = pq.top(); pq.pop();
9         pq.push(new HuffmanNode(-1, a->freq + b->freq));
10    }
11    // Kod tablosu olustur, bit dizisine cevir
12 }

```

Listing 2.3 Huffman kodlama (huffman.cpp)

2.3 Katsayı Kuantizasyonu

Kuantizasyon, wavelet katsayılarının belirli adımlarla yuvarlanmasını sağlar:

$$\hat{c} = \text{round} \left(\frac{c}{Q} \right) \cdot Q \quad (2.1)$$

Burada c orijinal katsayı, Q kuantizasyon adımı, $\hat{c}$ ise kuantize edilmiş katsayıdır. Q değeri arttıkça sıkıştırma oranı artar; ancak PSNR genellikle düşer.

Listing 2.4’de kuantizasyon ve hard thresholding fonksiyonlari verilmistir.

```

1 void quantize(Mat& coeff, double Q) {
2     for (int i = 0; i < coeff.rows; i++)
3         for (int j = 0; j < coeff.cols; j++)
4             coeff.at<double>(i, j) = round(coeff.at<double>(i, j) /
5                 Q) * Q;
6 }
7 void hardThreshold(Mat& coeff, double T) {
8     for (int i = 0; i < coeff.rows; i++)
9         for (int j = 0; j < coeff.cols; j++)
10             if (abs(coeff.at<double>(i, j)) < T)
11                 coeff.at<double>(i, j) = 0.0;
12 }

```

Listing 2.4 Kuantizasyon ve esikleme (wavelet.cpp)

2.4 Thresholding (Eşikleme)

Thresholding, mutlak değeri belirli bir eşiğin altında kalan katsayıları sıfırlayarak kodlama verimliliğini artırır.

2.4.1 Hard Thresholding

$$\hat{c} = \begin{cases} 0, & |c| < T \\ c, & |c| \geq T \end{cases} \quad (2.2)$$

2.4.2 Soft Thresholding

$$\hat{c} = \begin{cases} 0, & |c| < T \\ \text{sign}(c)(|c| - T), & |c| \geq T \end{cases} \quad (2.3)$$

Soft thresholding genellikle daha yumuşak görsel geçişler sağlar; hard thresholding ise daha agresif sıkıştırma sunar.

2.5 DWT ve Huffman Birlikte Kullanımı

Sistemde veri akışı aşağıdaki gibidir:

Görüntü → DWT → Quantization → Thresholding → Huffman → IDWT → Reconstructed Görüntü

Bu yapı, frekans domeninde enerji yoğunlaştırması ile istatistiksel kodlama avantajlarını birleştirir. Projede Huffman decode adımı ayrıca uygulanmamış; sıkıştırma performansı katsayıların kodlanmış boyutu üzerinden analiz edilmiştir. Rekonstrüksiyon doğrudan işlenmiş katsayılar üzerinden IDWT ile gerçekleştirilmiştir.

2.6 Değerlendirme Metrikleri

2.6.1 PSNR

Peak Signal-to-Noise Ratio (PSNR), rekonstrüksiyon kalitesini ölçmek için kullanılır:

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right) \quad (2.4)$$

$$MSE = \frac{1}{N} \sum_{i=1}^N (I_i - K_i)^2 \quad (2.5)$$

2.6.2 Sıkıştırma Oranı

Projede sıkıştırma oranı şu şekilde tanımlanmıştır:

$$CR = \frac{\text{Original Size}}{\text{Compressed Size}} \quad (2.6)$$

Bu tanımda daha yüksek CR değeri daha iyi sıkıştırma anlamına gelir.

3

Renkli Görüntü Temsili, YCbCr ve OpenCV Kullanımı

3.1 Dijital Görüntü Temsili

Dijital görüntüler piksel matrisleri olarak temsil edilir. Gri seviye görüntülerde her piksel tek kanallı yoğunluk değeri taşıırken, renkli görüntülerde her piksel genellikle üç kanaldan oluşur.

- **Gri seviye:** 1 kanal, 8-bit (0–255)
- **BGR/RGB:** 3 kanal, kanal başına 8-bit

OpenCV varsayılan olarak BGR sırasını kullanır. Projede kanal ayrıştırma işlemleri `cv::split` ve birleştirme işlemleri `cv::merge` fonksiyonları ile yapılmıştır.

3.2 RGB Kanal Bazlı Sıkıştırma

Projenin temel renkli görüntü genişletmesi, görüntünün B, G ve R kanallarına ayrılması ve her kanala bağımsız olarak aynı pipeline'ın uygulanmasıdır:

BGR Görüntü → **split** → **[B, G, R]** → **DWT + Quant + Threshold + Huffman**
→ **merge** → **Rekonstrüksiyon**

Bu yaklaşım uygulaması kolaydır ve her kanal için ayrı PSNR/sıkıştırma analizi yapılmasına imkân tanır. Ancak RGB uzayında kanallar arası korelasyon doğrudan kullanılmaz.

3.3 YCbCr Renk Uzayı

İnsan görsel sistemi parlaklığa (luminance) chrominance bileşenlerine göre daha duyarlıdır. Bu nedenle YCbCr uzayında luminance kanalı daha az, chrominance kanalları daha agresif sıkıştırılabilir.

Dönüşüm denklemleri:

$$Y = 0.299R + 0.587G + 0.114B \quad (3.1)$$

$$Cb = -0.169R - 0.331G + 0.500B + 128 \quad (3.2)$$

$$Cr = 0.500R - 0.419G - 0.081B + 128 \quad (3.3)$$

Ters dönüşüm ile tekrar BGR görüntüsü elde edilir. Projede Y kanalı için $Q = 1, T = 0$ profili; Cb ve Cr kanalları için $Q = 8, T = 5$ profili kullanılmıştır.

Listing 3.1’de RGB→YCbCr dönüşümü gösterilmektedir. Her piksel için Y , Cb , Cr bileşenleri ayrı matrislerde saklanır.

```
1 void rgbToYCbCr(const Mat& bgr, Mat& y, Mat& cb, Mat& cr) {  
2     for (int i = 0; i < bgr.rows; i++)  
3         for (int j = 0; j < bgr.cols; j++) {  
4             Vec3b pixel = bgr.at<Vec3b>(i, j);  
5             double B = pixel[0], G = pixel[1], R = pixel[2];  
6             y.at<double>(i, j) = 0.299*R + 0.587*G + 0.114*B;  
7             cb.at<double>(i, j) = -0.169*R - 0.331*G + 0.500*B +  
                128.0;  
8             cr.at<double>(i, j) = 0.500*R - 0.419*G - 0.081*B +  
                128.0;  
9         }  
10 }
```

Listing 3.1 RGB’den YCbCr’ye donusum (color.cpp)

Listing 3.2’de kanal profilleri tanımlanmıştır. YCbCr modunda parlaklık kanalı korunurken, renk kanallarına daha agresif sıkıştırma uygulanır.

```
1 vector<ChannelProfile> getProfiles(ColorMode mode) {  
2     if (mode == GRAY) return {{10.0, 5.0}};  
3     if (mode == RGB) return {{10.0, 5.0}, {10.0, 5.0},  
        {10.0, 5.0}};  
4     return {{1.0, 0.0}, {8.0, 5.0}, {8.0, 5.0}};  
5 }
```

Listing 3.2 Kanal bazli sikistirma profilleri (color.cpp)

3.4 OpenCV Kullanımı

Görüntü okuma/yazma, kanal ayırıştırma, normalize etme ve görselleştirme işlemleri OpenCV 4.x kütüphanesi ile gerçekleştirilmiştir.

3.4.1 Görüntü Okuma

Gri ve renkli görüntüler OpenCV `imread` fonksiyonu ile okunur:

```
1 Mat gray = imread("lena.bmp", IMREAD_GRAYSCALE);  
2 Mat color = imread("lena.png", IMREAD_COLOR);
```

Listing 3.3 Görüntü okuma (`main.cpp`)

3.4.2 Kanal Ayırıştırma

Listing 3.4’de `splitChannels` fonksiyonu kanal ayırıştırmasını gösterir.

```
1 vector<Mat> splitChannels(const Mat& image, ColorMode mode) {  
2     vector<Mat> channels;  
3     if (mode == GRAY) { /* tek kanal */ return channels; }  
4     if (mode == RGB) { split(image, channels); return channels; }  
5     Mat y, cb, cr;  
6     rgbToYCbCr(image, y, cb, cr);  
7     return {y, cb, cr};  
8 }
```

Listing 3.4 Kanal ayrıştırma (`color.cpp`)

3.4.3 Görselleştirme

DWT alt bantlarının incelenmesi için `showLevel` fonksiyonu kullanılmıştır. Bu fonksiyon LL, LH, HL ve HH bileşenlerini normalize ederek 2×2 blok yapısında görselleştirir.

3.5 OpenCV’nin Projeye Katkısı

OpenCV kullanımı projeye şu avantajları sağlamıştır:

- Farklı görüntü formatlarının kolay okunması
- Kanal bazlı işleme altyapısı
- Mat tabanlı bellek yönetimi

- Rekonstrüksiyon görsellerinin hızlı kaydedilmesi

Bu sayede geliştirme süreci hızlanmış, deneysel analizler için görsel çıktılar otomatik üretilebilir hale gelmiştir.

4

Sistem Tasarımı

4.1 Genel Sistem Mimarisi

Geliştirilen sistem modüler bir mimari ile tasarlanmıştır. Ana bileşenler ve sorumlulukları Tablo 4.1’de özetlenmiştir.

Tablo 4.1 Sistem Modülleri

Modül	Sorumluluk
wavelet.cpp	DWT/IDWT, quantization, thresholding
huffman.cpp	Huffman encode
color.cpp	RGB/YCbCr dönüşümleri, kanal profilleri
metrics.cpp	PSNR, sıkıştırma oranı
compress.cpp	Uçtan uca pipeline
main.cpp	Komut satırı arayüzü
experiments.cpp	Toplu deney koşulları ve CSV çıktısı

4.2 Sıkıştırma Süreci

Tek kanal için sıkıştırma adımları:

1. Giriş kanalı CV_64F formatına dönüştürülür.
2. dwt2D ile çok seviyeli wavelet ayrıştırması uygulanır.
3. quantize fonksiyonu ile katsayı kuantizasyonu yapılır.
4. Hard veya soft thresholding uygulanır.
5. Katsayılar tamsayıya yuvarlanarak Huffman kodlamaya verilir.
6. idwt2D ile rekonstrüksiyon gerçekleştirilir.

7. PSNR ve sıkıştırma oranı hesaplanır.

Renkli görüntülerde bu süreç her kanal için tekrarlanır; ardından kanallar birleştirilerek renkli rekonstrüksiyon elde edilir.

Listing 4.1’de tek kanallı sıkıştırma pipeline’ı gösterilmektedir.

```
1 ChannelResult compressChannel(const Mat& channel, ...) {
2     Mat coeff = toWorkChannel(channel).clone();
3     dwt2D(coeff, wf, config.level);
4     quantize(coeff, profile.quantStep);
5     hardThreshold(coeff, profile.threshold);
6     result.encoded = huffmanEncode(data, root, code);
7     idwt2D(rec, wf, config.level);
8     result.psnr = channelPSNR(original8U, result.reconstructed);
9     return result;
10 }
```

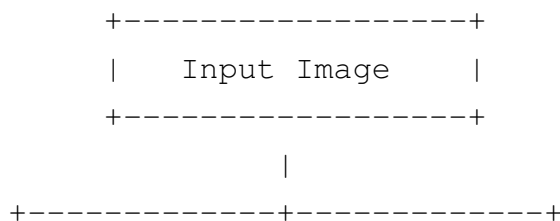
Listing 4.1 Tek kanal sıkıştırma (compress.cpp)

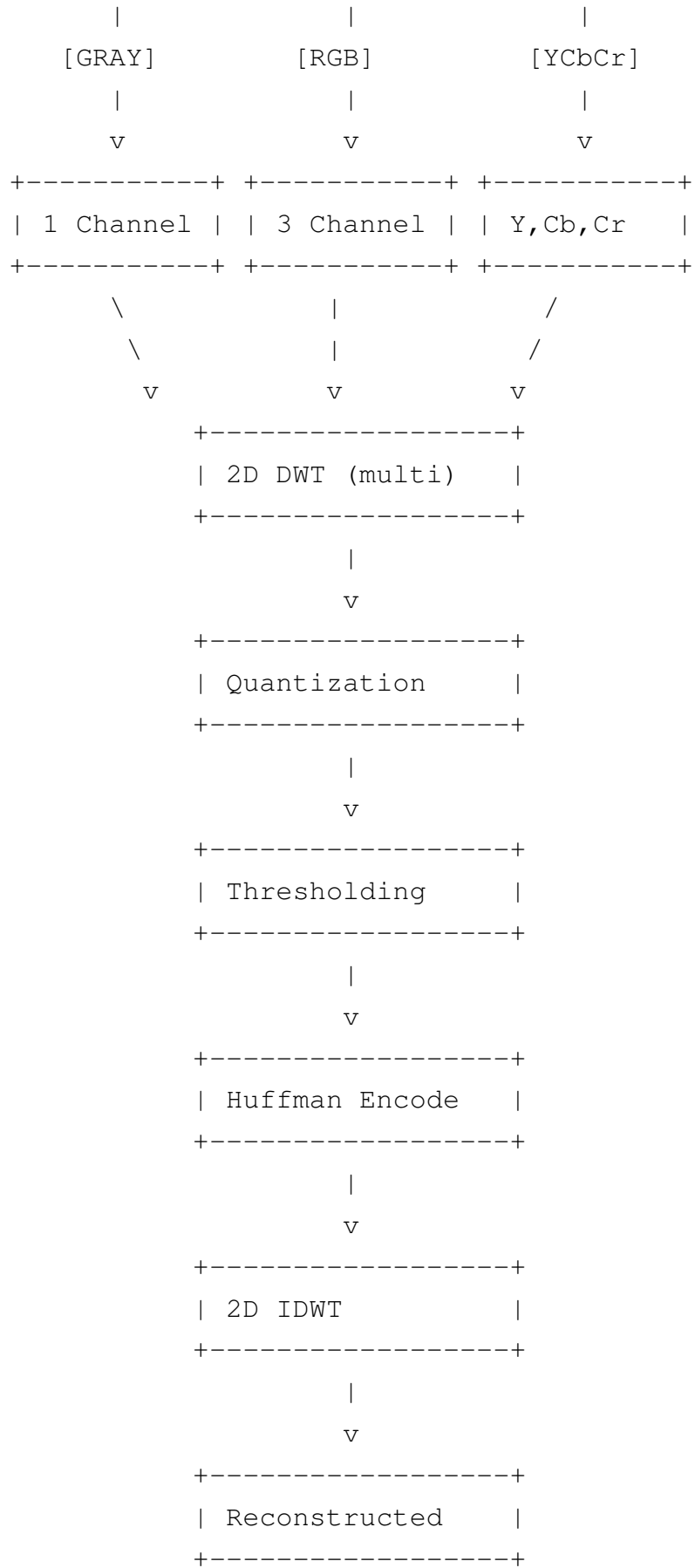
Listing 4.2’de renkli görüntü sıkıştırma fonksiyonu verilmiştir.

```
1 CompressionResult compressImage(const Mat& image, const
2     CompressionConfig& cfg) {
3     vector<Mat> inputChannels = splitChannels(image, cfg.colorMode
4         );
5     for (size_t i = 0; i < inputChannels.size(); i++) {
6         auto chResult = compressChannel(inputChannels[i], wf, cfg,
7             profiles[i]);
8         result.compressedBytes += chResult.compressedBytes;
9     }
10    result.reconstructed = mergeChannels(reconstructedChannels,
11        cfg.colorMode, ...);
12    return result;
13 }
```

Listing 4.2 Renkli görüntü sıkıştırma (compress.cpp)

4.3 Veri Akış Diyagramı





4.4 Parametre Yönetimi

Sistem davranışı aşağıdaki parametrelerle kontrol edilir:

- **wavelet:** HAAR, DB2, DB4, SYM4
- **level:** DWT ayrıştırma seviyesi (1, 2, 3, ...)
- **Q:** Kuantizasyon adımı
- **threshold:** Eşikleme değeri
- **color mode:** gray, rgb, ycbcr

Komut satırı arayüzü (wavelet_compress) ve toplu deney aracı (wavelet_experiments) bu parametreleri kullanarak farklı konfigürasyonları test eder.

Listing 4.3’de komut satırı arayüzü gösterilmektedir.

```
1 CompressionConfig config;  
2 config.wavelet = parseWavelet(opts.wavelet);  
3 config.colorMode = parseColorMode(opts.mode);  
4 Mat image = loadImage(opts.input, config.colorMode);  
5 CompressionResult result = compressImage(image, config);  
6 printResult(result);
```

Listing 4.3 Komut satiri arayuzu (main.cpp)

Listing 4.4’de otomatik deney aracı verilmiştir.

```
1 for (WaveletType w : {HAAR, DB2, DB4, SYM4}) {  
2     config.wavelet = w;  
3     rows.push_back(runSingle("D1-" + waveletName(w), ...));  
4 }  
5 writeCsv(rows, "results/deney_sonucclari.csv");
```

Listing 4.4 Toplu deney araci (experiments.cpp)

Listing 4.5’de PSNR hesaplama fonksiyonu gösterilmektedir.

```
1 double channelPSNR(const Mat& original, const Mat& reconstructed)  
2 {  
3     Mat diff; absdiff(original, reconstructed, diff);  
4     diff = diff.mul(diff);  
5     double mse = sum(diff)[0] / original.total();  
6     if (mse == 0.0) return 100.0;
```

```
6     return 10.0 * log10((255.0 * 255.0) / mse);  
7 }
```

Listing 4.5 PSNR hesaplama (`metrics.cpp`)

4.5 Modüler Tasarımın Avantajları

Modüler yapı sayesinde:

- Her algoritma bağımsız test edilebilir.
- Renk uzayı genişletmeleri ana pipeline'ı bozmadan eklenebilir.
- Deneysel analiz kodu üretim kodundan ayrılmıştır.
- Bakım ve raporlama süreçleri kolaylaşmıştır.

4.6 Performans Kriterleri

Sistem performansı aşağıdaki metriklerle değerlendirilir:

- PSNR (dB)
- Sıkıştırma oranı (CR)
- Sıfır katsayı oranı (%)
- Görsel rekonstrüksiyon kalitesi

Bu metrikler hem tekil koşullar hem de otomatik deney paketinde raporlanır.

4.7 Kaynak Kod Deposu

Projenin kaynak kodu, test görüntüleri, deney çıktıları ve birim testleri aşağıdaki GitHub deposunda yayımlanmıştır:

<https://github.com/muhtalipdede/MTM6107-wavelet>

Depo; `src/` altında modüler C++ implementasyonunu, `tests/` altında Google Test tabanlı birim testlerini, `results/` altında ise bu raporda kullanılan deneysel sonuçları içermektedir. Derleme ve çalıştırma adımları depodaki `README.md` dosyasında açıklanmıştır.

Bu bölümde deneysel sonuçlar yorumlanmakta; parametre seçimleri, yöntem karşılaştırmaları ve sistemin sınırları tartışılmaktadır.

5.1 Wavelet Seçimi

Deneylerde wavelet türü, hem PSNR hem de sıkıştırma oranını belirgin biçimde etkilemiştir. HAAR filtresi kısa olduğu için kuantizasyon sonrasında yapısal bilgiyi daha iyi korumuş ve 39.80 dB PSNR değerine ulaşmıştır. Buna karşılık SYM4 ve DB4, daha yüksek sıfır katsayı oranı sayesinde 3.9x civarında sıkıştırma sağlamıştır.

DB2 sonucu (12.95 dB PSNR, 2.01x sıkıştırma) diğer filtrelerden belirgin biçimde ayrılmaktadır. Bu durum, DB2 katsayılarının $Q = 10$ ve $T = 5$ parametreleri altında yeterince seyrekleşmemesinden kaynaklanmaktadır. DB2 için daha küçük kuantizasyon adımı veya farklı eşik değeri denendiğinde sonuçların iyileşebileceği değerlendirilmektedir. Pratik uygulamalarda hedef kaliteye göre wavelet seçimi yapılmalıdır: kalite öncelikli senaryolarda HAAR, denge arayan uygulamalarda SYM4 tercih edilebilir.

5.2 Decomposition Seviyesi

Ayrıştırma seviyesi arttıkça sıkıştırma oranı yükselirken PSNR düşmektedir. Level 1'den level 2'ye geçişte PSNR yaklaşık 4.7 dB azalırken sıkıştırma oranı %40 artmaktadır. Level 2'den level 3'e geçişte ise ek sıkıştırma kazancı sınırlı kalırken kalite kaybı devam etmektedir.

Bu nedenle test görüntüsü için level 2, kalite ve boyut arasında en uygun denge noktası olarak değerlendirilmiştir. Daha yüksek çözünürlüklü görüntülerde level 3'ün ek avantaj sağlayabileceği düşünülebilir; ancak mevcut deney setinde bu kazanç sınırlı kalmıştır.

5.3 Renk Uzayı Karşılaştırması

RGB yaklaşımı, her kanala aynı sıkıştırma profilini uygulayarak uygulaması kolay bir çözüm sunmaktadır. YCbCr yaklaşımında ise parlaklık kanalına (Y) daha hafif, renk farkı kanallarına (Cb , Cr) daha agresif profil uygulanmıştır.

Deneylerde her iki yöntem de benzer PSNR değerine ulaşmıştır (RGB: 19.66 dB, YCbCr: 19.68 dB). Bu parametre setinde RGB modu 3.75x sıkıştırma oranı ile YCbCr'den (2.93x) daha yüksek boyut kazancı sağlamıştır. Bunun nedeni, YCbCr profilinde Y kanalının $Q = 1$, $T = 0$ ile korunması ve toplam sıkıştırılmış boyutun artmasıdır.

YCbCr yaklaşımının değeri yalnızca ham sıkıştırma oranında değil, algısal kaliteyi önceliklendiren kanal bazlı profillemeye görülmektedir. Chrominance kanallarına agresif profil uygulanırken parlaklık bilgisi daha az bozulmaktadır.

5.4 Algoritmik İyileştirmelerin Birlikte Etkisi

Kuantizasyon ve eşikleme tek başına değil, birlikte uygulandığında anlamlı sonuç vermiştir. Kuantizasyon katsayı dağılımını sınırlarken, eşikleme küçük katsayıları sıfırlayarak Huffman kodlamanın verimliliğini artırmaktadır. Deneylerde sıfır katsayı oranı %63–80 aralığına çıkmış; bu da 2.8x–4.6x sıkıştırma oranlarının elde edilmesine katkı sağlamıştır.

Multi-level DWT ise frekans ayrımını güçlendirerek aynı Q ve T değerleri altında daha yüksek seyreklik sağlamıştır. Parametrelerin sabit kodlanmaması, farklı görüntü türleri için ayar yapılabilmesine olanak tanımaktadır.

5.5 JPEG ile Karşılaştırma

Geliştirilen sistem, DWT tabanlı bir sıkıştırma pipeline'ı kullanırken JPEG standardı DCT tabanlı blok sıkıştırmaya dayanmaktadır [5]. JPEG yaygın ve hızlı bir format olmasına karşın, yüksek sıkıştırma oranlarında blok artefaktları oluşturabilir. Wavelet tabanlı yöntemler ise çok çözünürlüklü temsil sayesinde daha yumuşak bozulmalar üretebilir.

Bu projede tam JPEG implementasyonu yapılmamıştır; dolayısıyla doğrudan bit düzeyinde karşılaştırma mümkün değildir. Yine de yöntemsel fark, wavelet tabanlı yaklaşımın frekans bantları üzerinde daha esnek kontrol sağladığını göstermektedir.

5.6 Sistem Sınırlamaları

Geliştirilen sistemde aşağıdaki sınırlamalar bulunmaktadır:

- Huffman kodlama yalnızca sıkıştırılmış boyut hesabı için kullanılmaktadır; decode adımı uygulanmamıştır. Rekonstrüksiyon doğrudan kuantize edilmiş katsayılar üzerinden yapılmaktadır.
- Sıkıştırma oranı, Huffman çıktısının byte boyutuna göre hesaplanmaktadır; gerçek bir dosya formatı (header, metadata) kullanılmamaktadır.
- DWT implementasyonu periodik sınır koşulu kullanmaktadır; görüntü kenarlarında halka artefaktları oluşabilir.
- Deneyler yalnızca iki test görüntüsü üzerinde yapılmıştır; farklı içerik türlerinde sonuçlar değişebilir.
- Gerçek zamanlı veya GPU hızlandırmalı işleme desteklenmemektedir.

5.7 Kalite–Sıkıştırma Dengesi

Deneyler genel olarak yüksek sıkıştırma ile yüksek kalite arasında ters bir ilişki olduğunu göstermiştir. Q ve T değerlerinin artırılması boyutu küçültürken PSNR'ı düşürmektedir. Level artışı da benzer etkiyi yapmaktadır. YCbCr modunda renk kanallarına agresif profil uygulanması ise algılanan kaliteyi büyük ölçüde korurken dosya boyutunu azaltmaktadır.

Sonuç olarak sistem, ders kapsamındaki gereksinimleri karşılayan çalışır bir wavelet tabanlı sıkıştırma altyapısı sunmaktadır. Gelecek çalışmalarda Huffman decode, SPIHT veya EZW gibi gelişmiş kodlama yöntemleri ve daha geniş test seti ile performansın artırılabilceği değerlendirilmektedir.

6.1 Test Ortamı ve Deney Protokolü

Deneyisel analizler aşağıdaki ortamda gerçekleştirilmiştir:

- İşletim Sistemi: macOS
- Derleyici: Apple Clang (C++17)
- Kütüphane: OpenCV 4.13.0
- Derleme: CMake
- Gri test görüntüsü: `lena.bmp` (512×512, 262144 byte)
- Renkli test görüntüsü: `lena.png` (512×512, 786432 byte)
- Deney aracı: `./build/wavelet_experiments results`

Tüm deneyler otomatik olarak `wavelet_experiments` aracı ile koşturulmuş; sonuçlar `results/deney_sonucлари.csv` dosyasına, rekonstrüksiyon görselleri ise `results/images/` klasörüne kaydedilmiştir. Toplam 12 deney koşusu gerçekleştirilmiştir.

6.2 Sabit Deney Parametreleri

Karşılaştırmaların adil olması için aşağıdaki varsayılan parametreler kullanılmıştır:

- Kuantizasyon adımı: $Q = 10$
- Eşikleme: $T = 5$ (hard thresholding)
- Wavelet (level testleri hariç): SYM4

- Ayırıştırma seviyesi (wavelet testleri hariç): level = 2

YCbCr modunda renk profili şu şekilde ayarlanmıştır: Y kanalı için $Q = 1, T = 0$; Cb ve Cr kanalları için $Q = 8, T = 5$.

6.3 D1: Wavelet Türü Karşılaştırması

Tablo 6.1, `lena.bmp` gri görüntüsü üzerinde dört wavelet türünün performansını göstermektedir.

Tablo 6.1 Wavelet Türü Karşılaştırması (`lena.bmp`, Level=2, Q=10, T=5)

Wavelet	PSNR (dB)	CR (x)	Sıfır (%)	Orijinal (KB)	Sıkıştırılmış (KB)
HAAR	39.80	3.57	68.74	256.0	71.6
DB2	12.95	2.01	6.19	256.0	127.3
DB4	13.79	3.93	74.79	256.0	65.2
SYM4	19.37	3.96	75.16	256.0	64.6

6.3.1 D1 Sonuçlarının Yorumu

HAAR wavelet, 39.80 dB PSNR ile en yüksek görsel kaliteyi sağlamıştır. Bunun nedeni, Haar filtresinin kısa olması ve kuantizasyon sonrasında bile yapısal bilginin büyük ölçüde korunmasıdır. Ancak sıkıştırma oranı (3.57x), SYM4 ve DB4'e göre daha düşüktür; çünkü sıfır katsayı oranı (%68.74) diğer filtrelerden düşüktür.

SYM4 ve **DB4**, sıkıştırma açısından en iyi sonuçları vermiştir (sırasıyla 3.96x ve 3.93x). Sıfır katsayı oranlarının %74–75 civarında olması, thresholding sonrası Huffman kodlayıcının daha verimli çalışmasını sağlamıştır. SYM4, görüntü işleme uygulamalarında yaygın kullanılan simetrik yapısı sayesinde DB4'e göre biraz daha yüksek PSNR (19.37 dB) elde etmiştir.

DB2 ise bu parametreler altında beklenmedik şekilde kötü performans göstermiştir: PSNR yalnızca 12.95 dB, sıkıştırma oranı 2.01x ve sıfır katsayı oranı %6.19. Bu durum, DB2 filtresinin `lena.bmp` üzerinde eşikleme ve kuantizasyon sonrası katsayıları yeterince seyreltmediğini; dolayısıyla hem kalite hem sıkıştırma açısından verimsiz kaldığını göstermektedir. Bu nedenle DB2, bu görüntü ve parametre seti için uygun bir seçim değildir.



Şekil 6.1 Wavelet karşılaştırması: HAAR (sol), SYM4 (orta), DB2 (sağ)

Şekil 6.1’de görüldüğü üzere HAAR rekonstrüksiyonu en temiz sonucu verirken, DB2 ciddi bozulma ve blok artefaktları üretmiştir.

6.4 D2: Decomposition Seviyesi Karşılaştırması

Tablo 6.2, SYM4 wavelet ile farklı ayrıştırma seviyelerinin etkisini göstermektedir.

Tablo 6.2 Decomposition Seviyesi Karşılaştırması (lena.bmp, SYM4, Q=10, T=5)

Level	PSNR (dB)	CR (x)	Sıfır (%)	Orijinal (KB)	Sıkıştırılmış (KB)
1	24.09	2.82	63.33	256.0	90.6
2	19.37	3.96	75.16	256.0	64.6
3	16.21	4.49	76.87	256.0	57.0

6.4.1 D2 Sonuçlarının Yorumu

Ayrıştırma seviyesi arttıkça belirgin bir **kalite–sıkıştırma trade-off** gözlemlenmiştir:

- **Level 1:** En yüksek PSNR (24.09 dB), ancak en düşük sıkıştırma (2.82x). Orijinal 256 KB’lık görüntü 90.6 KB’a düşmüştür. Görsel kalite iyi, fakat sıkıştırma sınırlıdır.
- **Level 2:** PSNR 19.37 dB’ye düşerken sıkıştırma 3.96x’e yükselmiştir. Sıfır katsayı oranı %63’ten %75’e çıkmıştır. Bu konfigürasyon kalite ve verimlilik arasında dengeli bir noktadır.
- **Level 3:** En yüksek sıkıştırma (4.49x, 57 KB), ancak PSNR 16.21 dB ile en düşük kalite. Görüntüde belirgin yumuşama ve detay kaybı oluşmaktadır.

Level 1'den Level 2'ye geçişte PSNR yaklaşık 4.7 dB düşerken sıkıştırma oranı %40 artmaktadır ($2.82x \rightarrow 3.96x$). Level 2'den Level 3'e geçişte ise PSNR 3.2 dB daha düşerken sıkıştırma yalnızca %13 artmaktadır ($3.96x \rightarrow 4.49x$). Bu, level 3'ün ek sıkıştırma kazancının sınırlı olduğunu; buna karşılık kalite kaybının belirginleştiğini göstermektedir.



Şekil 6.2 Seviye karşılaştırması: Level 1 (sol), Level 2 (orta), Level 3 (sağ)

6.5 D3: Gri ve Renkli Görüntü Karşılaştırması

Tablo 6.3, aynı SYM4/level=2 parametreleriyle gri ve renkli sıkıştırma sonuçlarını özetlemektedir.

Tablo 6.3 Gri ve Renkli Görüntü Karşılaştırması (SYM4, Level=2, Q=10, T=5)

Mod	Görüntü	PSNR (dB)	CR (x)	Orijinal (KB)	Sıkıştırılmış (KB)
Gray	lena.bmp	19.37	3.96	256.0	64.6
RGB	lena.png	19.66	3.75	768.0	204.8
YCbCr	lena.png	19.68	2.93	768.0	262.4

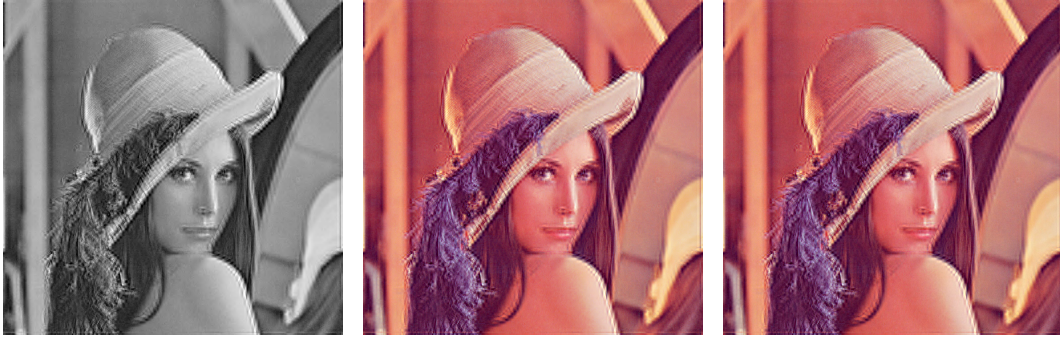
6.5.1 D3 Sonuçlarının Yorumu

Gri ve renkli modların PSNR değerleri birbirine çok yakındır (19.37–19.68 dB). Bu, renkli görüntü desteğinin eklenmesinin görsel kaliteyi anlamlı biçimde bozmadığını göstermektedir.

Sıkıştırma boyutları moda göre değişmektedir:

- Gri görüntü: 256 KB $\rightarrow$ 64.6 KB (tek kanal)
- RGB: 768 KB $\rightarrow$ 204.8 KB (üç kanal, eşit profil; kanal başına ~ 68.3 KB)
- YCbCr: 768 KB $\rightarrow$ 262.4 KB ($Y: Q = 1, T = 0; Cb/Cr: Q = 8, T = 5$)

Bu deney setinde RGB modu daha yüksek sıkıştırma oranı sağlamıştır. YCbCr modunda Y kanalının korunması, chrominance kanallarındaki agresif sıkıştırmaya rağmen toplam boyutu artırmıştır. Yine de YCbCr yaklaşımı, algısal kaliteyi önceliklendiren uygulamalarda tercih edilebilir.



Şekil 6.3 Gri (sol), RGB (orta) ve YCbCr (sağ) rekonstrüksiyon karşılaştırması

6.6 D4: RGB ve YCbCr Karşılaştırması

Tablo 6.4, aynı renkli görüntü üzerinde iki renk uzayı yaklaşımını doğrudan karşılaştırır.

Tablo 6.4 RGB ve YCbCr Karşılaştırması (lena.png, SYM4, Level=2)

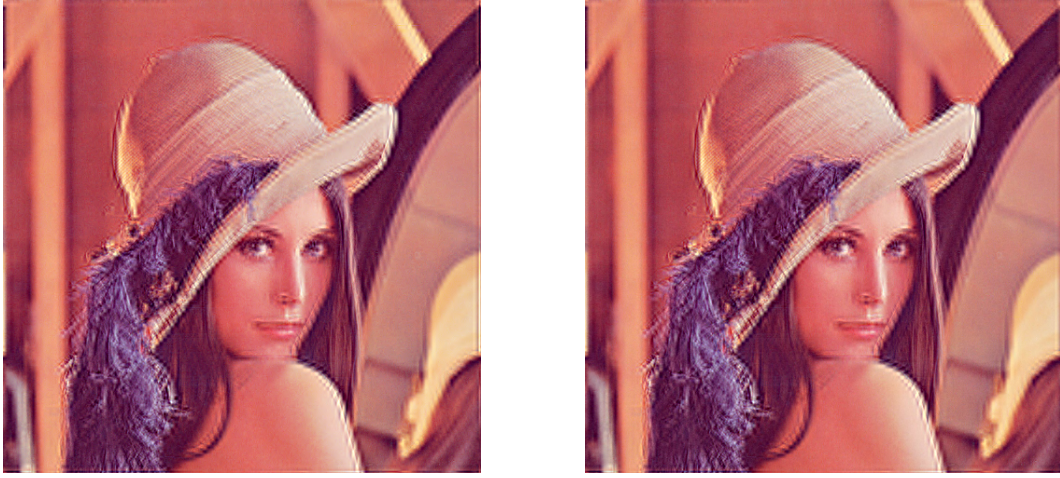
Renk Uzayı	PSNR (dB)	CR (x)	Sıfır (%)	Sıkıştırılmış (KB)
RGB	19.66	3.75	68.73	204.8
YCbCr	19.68	2.93	59.68	262.4

6.6.1 D4 Sonuçlarının Yorumu

PSNR değerleri birbirine çok yakındır (19.66–19.68 dB). Sıkıştırma açısından ise RGB modu bu parametreler altında daha verimli sonuç vermiştir:

- RGB modunda tüm kanallara $Q = 10$, $T = 5$ uygulanmış; sıkıştırma oranı 3.75x olmuştur.
- YCbCr modunda Y kanalı $Q = 1$, $T = 0$ ile korunmuş; Cb/Cr kanalları $Q = 8$, $T = 5$ ile sıkıştırılmıştır.
- Y kanalının hafif profile korunması, toplam sıkıştırılmış boyutu 262.4 KB'a çıkarmıştır (RGB: 204.8 KB).
- YCbCr'de ortalama sıfır katsayı oranı %59.68, RGB'de %68.73'tür.

Bu sonuç, kanal profillemesinin yalnızca boyut değil algısal kalite odaklı bir tercih olduğunu göstermektedir. Daha agresif chrominance profili veya Y kanalı için ara parametreler denendiğinde farklı denge noktaları elde edilebilir.



Şekil 6.4 RGB (sol) ve YCbCr (sağ) rekonstrüksiyon karşılaştırması

6.7 Tüm Deneylerin Özet Tablosu

Tablo 6.5, 12 deney koşusunun tamamını tek tabloda özetlemektedir.

Tablo 6.5 Tüm Deneysel Sonuçların Özeti (wavelet_experiments)

ID	Grup	Wavelet	Level	Mode	PSNR (dB)	CR (x)
D1-HAAR	Wavelet	HAAR	2	gray	39.80	3.57
D1-DB2	Wavelet	DB2	2	gray	12.95	2.01
D1-DB4	Wavelet	DB4	2	gray	13.79	3.93
D1-SYM4	Wavelet	SYM4	2	gray	19.37	3.96
D2-L1	Level	SYM4	1	gray	24.09	2.82
D2-L2	Level	SYM4	2	gray	19.37	3.96
D2-L3	Level	SYM4	3	gray	16.21	4.49
D3-gray	Renk	SYM4	2	gray	19.37	3.96
D3-rgb	Renk	SYM4	2	rgb	19.66	3.75
D3-ycbcr	Renk	SYM4	2	ycbcr	19.68	2.93
D4-rgb	Renk Uz.	SYM4	2	rgb	19.66	3.75
D4-ycbcr	Renk Uz.	SYM4	2	ycbcr	19.68	2.93

6.8 Deneylerin Tekrarlanması

Kaynak kod ve deney betikleri GitHub deposunda yer almaktadır:

<https://github.com/muhtalipdede/MTM6107-wavelet>

Depo klonlandıktan sonra aşağıdaki komutlarla derleme ve deneyler çalıştırılabilir:

```
cmake -S . -B build
cmake --build build
./build/wavelet_experiments results
```

Çıktılar `results/deney_sonucлари.csv` dosyasına ve `results/images/` klasörüne kaydedilir.

7 Sonuç

Bu çalışmada wavelet tabanlı görüntü sıkıştırma sistemi geliştirilmiş; `lena.bmp` ve `lena.png` üzerinde 12 deney koşusu ile PSNR, sıkıştırma oranı ve sıfır katsayı oranı karşılaştırılmıştır. Elde edilen bulgular aşağıda özetlenmiştir.

Wavelet türü (D1): HAAR en yüksek PSNR değerini (39.80 dB) vermiştir. SYM4, 19.37 dB PSNR ve 3.96x sıkıştırma oranı ile en dengeli sonucu sağlamıştır. DB2, 12.95 dB ve 2.01x ile en düşük performansı göstermiştir.

Decomposition seviyesi (D2): Seviye arttıkça sıkıştırma oranı yükselirken PSNR düşmüştür (level 1: 24.09 dB / 2.82x; level 2: 19.37 dB / 3.96x; level 3: 16.21 dB / 4.49x). Level 2, kalite ve sıkıştırma arasında en uygun dengeyi sunmuştur.

Renk modu (D3–D4): Renkli işleme gri sonuca göre belirgin kalite kaybına yol açmamıştır (gri: 19.37 dB, RGB: 19.66 dB, YCbCr: 19.68 dB). RGB modu 3.75x sıkıştırma oranı ile en yüksek boyut kazancını sağlamıştır. YCbCr modu ise *Y* kanalını koruyan profille benzer PSNR sunmuş; toplam sıkıştırma oranı 2.93x olmuştur.

Genel olarak geliştirilen pipeline; gri görüntüde 256 KB veriyi 64.6 KB'a (3.96x), renkli görüntüde RGB ile 768 KB veriyi 204.8 KB'a (3.75x) indirebilmiştir. Deney sonuçları, wavelet dönüşümü, kuantizasyon, eşikleme ve Huffman kodlama adımlarının birlikte etkin çalıştığını göstermektedir.

Proje kaynak kodu ve deney çıktıları <https://github.com/muhtalipdede/MTM6107-wavelet> adresinde erişilebilir durumdadır.

- [1] A. N. Skodras, C. A. Christopoulos T. Ebrahimi, “The JPEG 2000 Still Image Compression Standard,” *IEEE Signal Processing Magazine*, c. 18, no. 5, ss. 36–58, 2001.
- [2] S. Mallat, *A Wavelet Tour of Signal Processing*, 2. bs. San Diego: Academic Press, 1999.
- [3] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, c. 40, no. 9, ss. 1098–1101, 1952.
- [4] I. Daubechies, *Ten Lectures on Wavelets*. Philadelphia: SIAM, 1992.
- [5] G. K. Wallace, “The JPEG Still Picture Compression Standard,” *IEEE Transactions on Consumer Electronics*, c. 38, no. 1, ss. 18–34, 1992.